实验 4 定时器

一. 实验目标

1.1. 完成一个通用定时器的初始化，并使用中断完成定时器计时任务

用通用定时器制作一个延时函数，以定时器更新事件为中断源，在中断服务函数中控制 LED 灯，需完成软件仿真和开发板实验。

1.2. 使用输入捕获功能记录某个按钮被按下的时间

需完成软件仿真和开发板实验。

1.3. 完成呼吸灯功能

需完成软件仿真和开发板实验。

二. 实验原理

2.1. 时基单元初始化

通过 STM32 参考手册，可知通用定时器 TIM2~TIM7 的时钟全都位于 APB1 总线，应使用 RCC_APB1PeriphClockCmd 使能时钟。

使用 TIM_TimeBaseInit 函数完成定时器时基单元的初始化，此前需要先对 TIM_TimeBaseInitTypeDef 类型结构体变量初始化。

使用 TIM_Cmd 使能定时器。

2.2. 中断初始化

使用 TIM_ITConfig 函数，使能定时器的更新中断。使用 NVIC_Init，完成定时器中断通道的设定。

通用定时器的中断服务函数为 TIMx_IRQHandler，其中 x 应为 2~7。在中断函数中通过 TIM_GetITStatus 获取中断源信息，如触发的是定时器更新中断，则完成开关 LED 灯的操作，并使用 TIM_ClearITPendingBit 清除中断标志位。

2.3. 输入捕获模式

完成时基单元初始化的基础上，使用 TIM_ICInit 函数完成对捕获/比较寄存器功能的初始化，此前需要先对 TIM5_ICInitStructure 类型结构体变量初始化。

找到按钮 KEY_UP 对应的 GPIO 口 PA0，并初始化 PA0，设为默认下拉的输入模式。

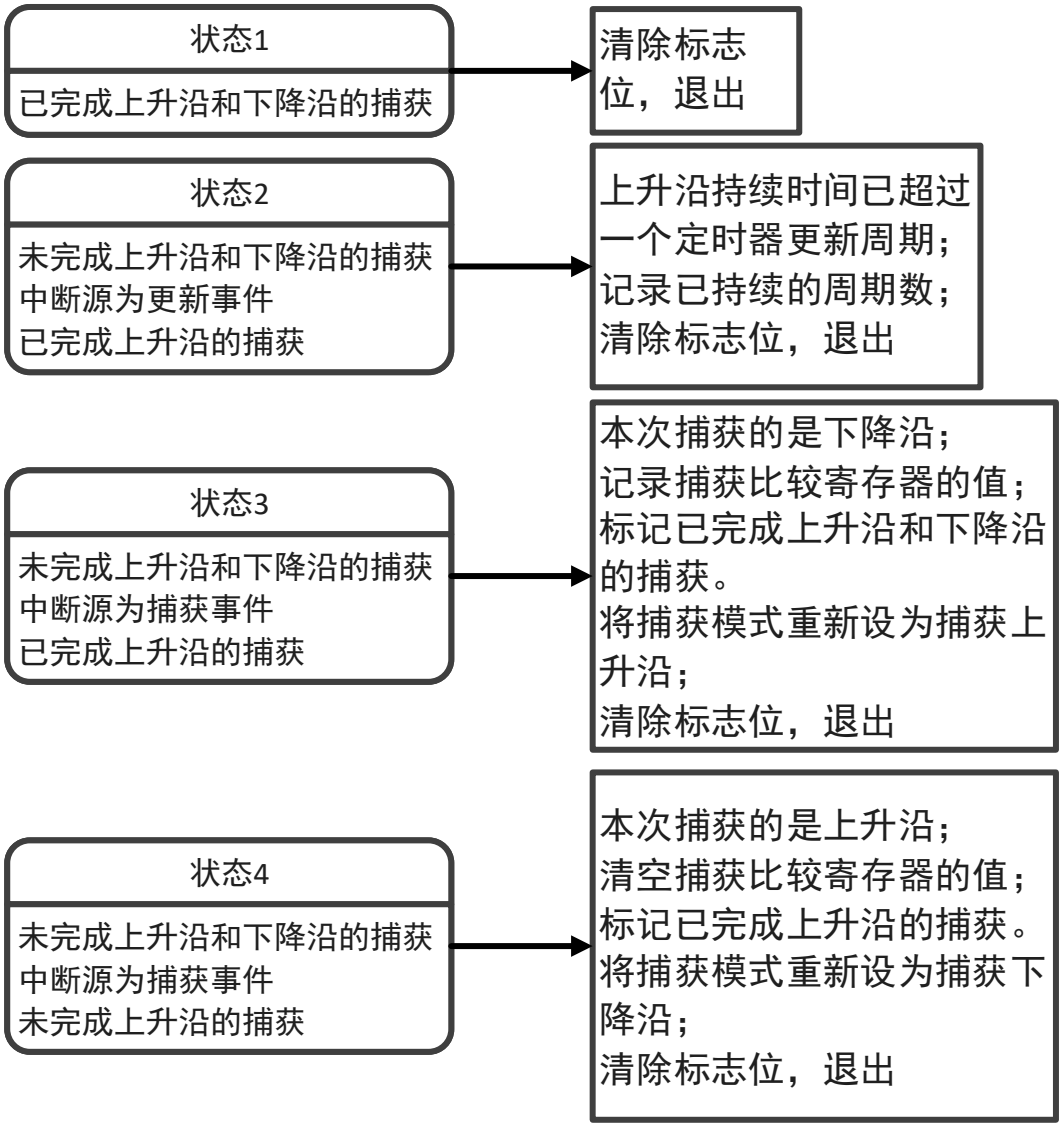
使用 TIM_ITConfig 初始化中断时，应同时使能定时器更新中断 TIM_IT_Update

和捕获中断 TIM_IT_CC1。

2.4. 识别按钮被按下和松开

根据原理图，KEY_UP 被按下时，PA0 由低电平转为高电平，被松开时，PA0 由高电平转为低电平。因此，只需要捕获到 PA0 上电平的上升沿，即可认为 KEY_UP 被按下了；捕获到 PA0 上的电平下降沿，即可认为 KEY_UP 被松开了。

按照此思路，可在中断服务函数中使用 TIM_GetITStatus 获取当前定时器中断的触发源，编写代码完成对下列状态的处理。



在 main 函数的主循环中检测用于记录“已完成上升沿和下降沿捕获”的标志位，如以标记为捕获完成，则根据上升沿持续的周期数和记录的捕获比较寄存器值来计算 PA0 上的高电平持续的时间。完成输出后，将所有标记位和用于记录的变量清零，以开始下一次采集。

2.5. PWM 输出模式

通过 STM32 参考手册和原理图可知，TIM3 定时器通道 2（TIM3_CH2）可重映射至 LED0 对应的 GPIO 口 PB5 上。因此，可使用定时器 3 的 PWM 输出模式，将产生的 PWM 波利用通道 2 输出到 LED0 上，控制 LED0 的明暗。

在完成时基单元初始化的基础上，使用 TIM_OC2Init 函数完成对于定时器 PWM 模式输出通道 2 的初始化。

在主函数的主循环中，使用 TIM_SetCompare2 可设定 PWM 输出波的占空比，以调整 LED0 明暗状态，动态调整 LED0 明暗状态以达到呼吸灯的效果。

2.6. 可能使用到的库函数

函数名	功能
RCC_APB2PeriphClockCmd	使能 APB1 上的时钟
RCC_APB2PeriphClockCmd	使能 APB2 上的时钟
GPIO_Init	使用 GPIO_InitTypeDef 结构体初始化 GPIO
TIM_TimeBaseInit	使用 TIM_TimeBaseInitTypeDef 结构体初始化定时器
TIM_ITConfig	使能定时器中断源
TIM_Cmd	使能定时器
NVIC_Init	使用 NVIC_InitTypeDef 结构体初始化 NVIC
TIMx_IRQHandler	待实现的中断服务函数，x=2~7
TIM_ClearITPendingBit	清除 TIMx 上挂起的中断标志位
TIM_GetITStatus	获取当前定时器中断源
TIM_ICInit	使用 TIM_ICInitTypeDef 结构体初始化捕获比较寄存器
TIM_OC1PolarityConfig	设定定时器通道 1 上输入捕获模式的捕获极性
TIM_OC2Init	初始化定时器输出通道 2
TIM_OC2PreloadConfig	使能定时器输出通道 2 的预装载寄存器
TIM_SetCompare2	设定定时器捕获比较寄存器 2 的值

三. 实验步骤

3.1. 任务一

3.1.1. 编写 timer.c

完成定时器初始化函数 TIM3_Int_Init，将用于控制定时器更新频率的 TIM_Period 和 TIM_Prescaler 作为函数的输入参数。定时器更新频率（延时时间，或产生更新事件/中断的时间间隔）可以利用下式计算：

延时时间=（TIMx_ARR+1）X（TIMx_PSC+1）/TIMxCLK

式中，TIMx_ARR 被赋值为 TIM_Period 成员变量的值，TIMx_PSC 被赋值为 TIM_Prescaler 成员变量的值，TIMxCLK 的值为 72 MHz（单位为 Tick/s，仅对于本实验开发板来说）

根据以上说明，实现定时器初始化函数。需依次完成变量声明、时钟使能、时基单元初始化和中断初始化等功能，最后使能定时器。

3.1.2. 实现中断服务函数 TIM3_IRQHandler

使用 TIM_GetITStatus 函数获取当前定时器的中断源，判断当前中断源是否为定时器更新中断 TIM_IT_Update，如果是，则切换 LED 灯的开/关状态，随后使用 TIM_ClearITPendingBit 清除中断标志位。

3.1.3. 仿真并下载运行

3.2. 任务二

3.2.1. 重新实现定时器初始化函数，需依次完成变量声明、时钟使能、时基单元初始化、**输入捕获**和中断初始化等功能，最后使能定时器。

3.2.2. 实现中断服务函数，根据状态图，完成记录 PA0 上高电平持续时间（以 tick 为单位）的功能。

3.2.3. 更新 main 函数，计算 PA0 上高电平持续时间（以 ms 为单位），输出后重置定时器和记录变量的状态。

3.3. 任务三

3.3.1. 重新实现定时器初始化函数，需依次完成变量声明、时钟使能、**GPIO 口重映射**、时基单元初始化、**PWM 模式输出、预装载寄存器初始化**等功能，最后使能定时器。

3.3.2. 更新 main 函数，使用 TIM_SetCompare2 不断调整 PWM 输出波的占空比，控制占空比由小到大、再由大变小。